

19. ROS2 Launch File Configuration

Note: The accompanying reference video uses the ROS Humble version, for reference only

1. Launch Introduction

So far, every time we run a ROS node, we need to open a new terminal and run a command. There are many nodes in a robot system, and it's troublesome to start them this way every time. Is there a way to start all nodes at once? The answer is definitely yes - that's the Launch file. It's a script for multi-node startup and configuration in the ROS system.

In ROS2, launch is used for multi-node startup and configuring program runtime parameters. ROS2 launch files support XML, YAML, and Python formats. This section uses Python format launch files as an example. Compared to the other two formats, Python format is more flexible:

- Python has many function libraries that can be used in launch files;
- ROS2 common launch features and specific launch features are written in Python, so you can access launch features that may not be exposed in XML and YAML;

When writing ROS2 launch files in Python, the main thing is to abstract each node, file, script, etc. into an action, and use a unified interface to start them.

References:

- Launch system design document: [ROS 2 Launch System Design Document](#)
- Launch official API documentation: [Launch API Documentation \(Updated with Official\)](#)
- Preparation: Create a package to store program files:

```
ros2 pkg create learn_launch --build-type ament_python
```

2. Writing Launch for a Single Node

2.1. Create Launch File

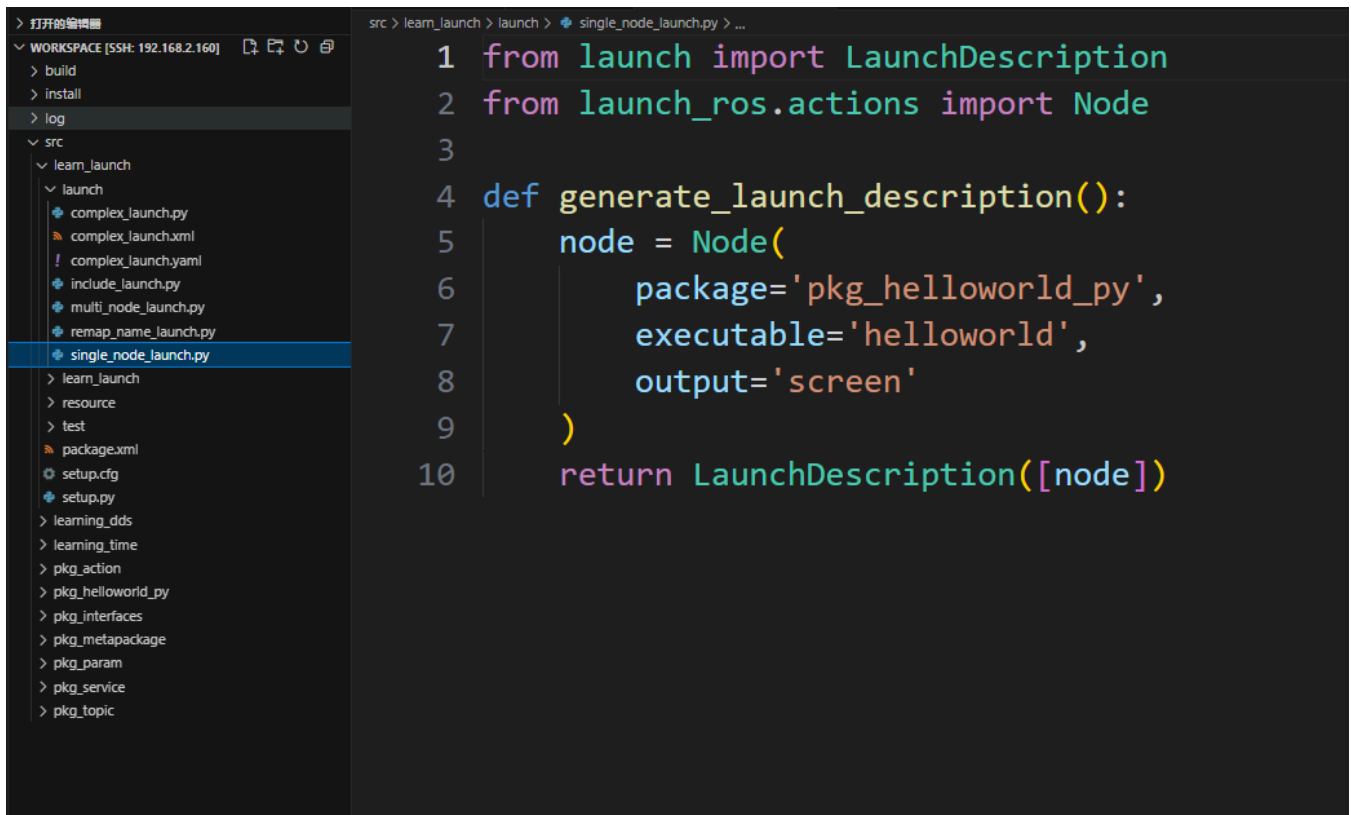
Create a launch folder under the package, then create a [single_node_launch.py] file in the launch folder. Copy the following content to this file:

```

from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    node = Node(
        package='pkg_helloworld_py',
        executable='helloworld',
        output='screen'
    )
    return LaunchDescription([node])

```



2.2. Configure setup.py File

Launch files are commonly named LaunchName_launch.py, where LaunchName is customizable and _launch.py is commonly considered fixed. You need to modify the setup.py file under the package. The modification is to add the launch path and files under it, so the build can generate executable .py files.

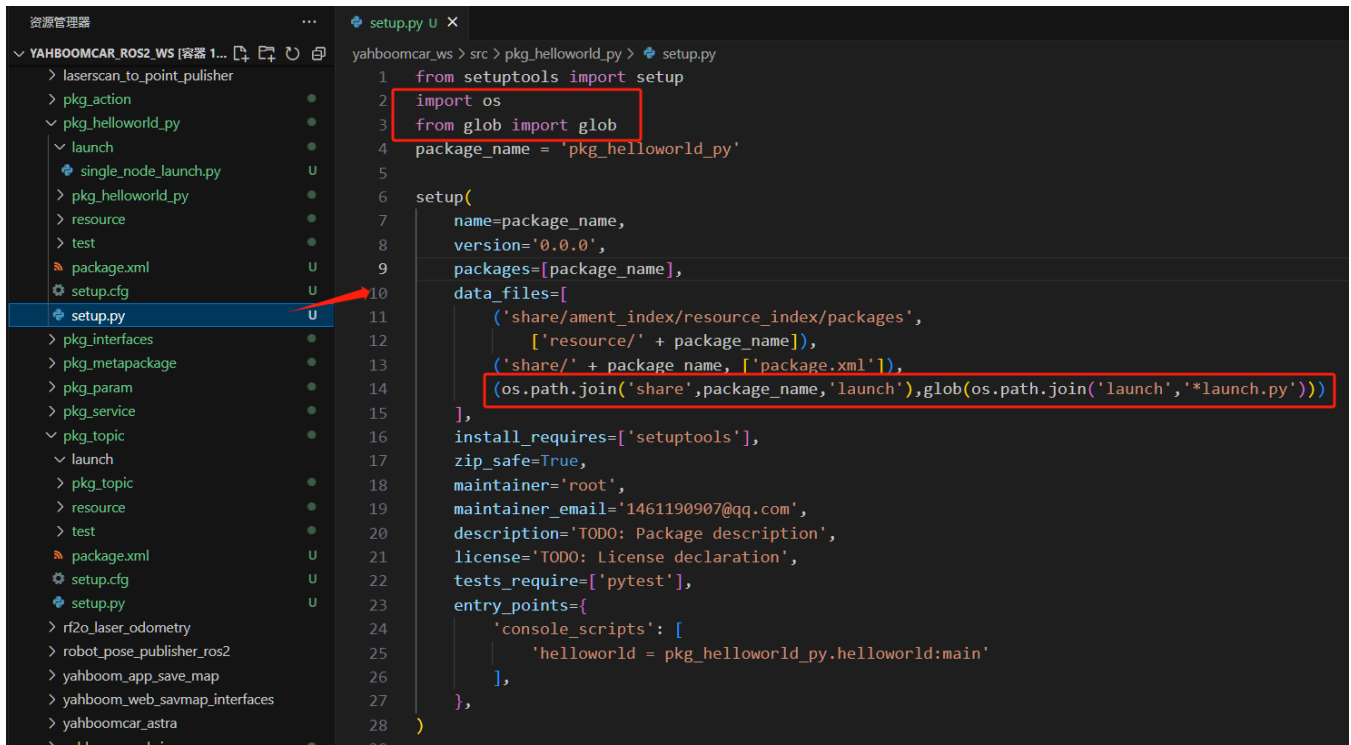
#1. Import related header files

```

import os
from glob import glob

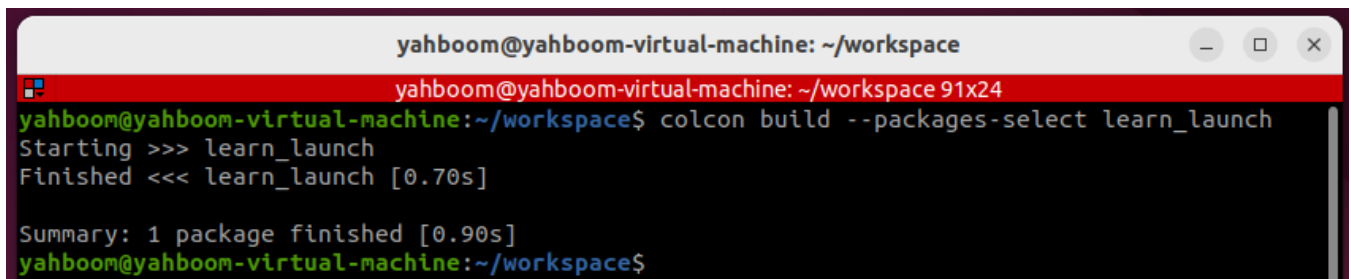
```

#2. In the data_files list, add the launch path and the launch.py file under the path (os.path.join('share',package_name,'launch'),glob(os.path.join('launch','*launch.py')))



2.3. Build Package

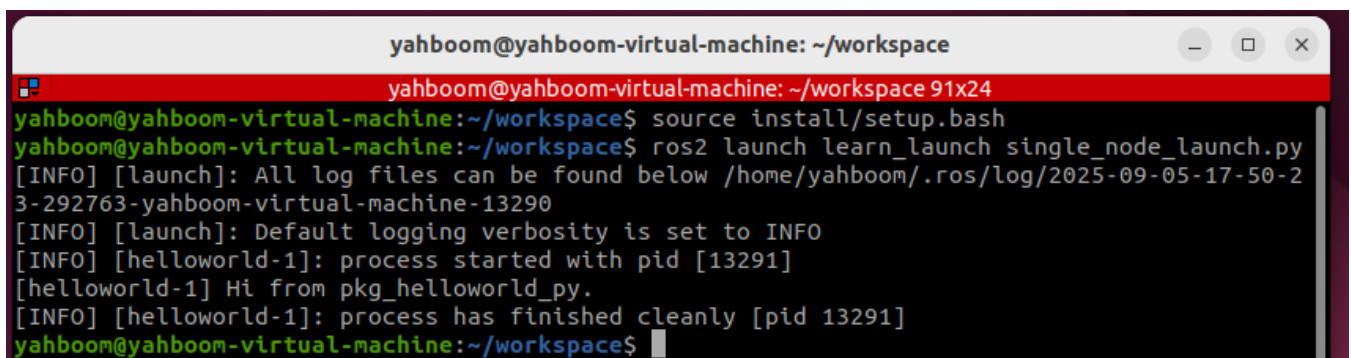
```
colcon build --packages-select learn_launch
```



2.4. Run Program

- Refresh environment variables, then run the launch file:

```
ros2 launch learn_launch single_node_launch.py
```



2.5. Source Code Analysis

1. Import related libraries:

```
from launch import LaunchDescription
from launch_ros.actions import Node
```

2. Define a function generate_launch_description that returns a launch_description:

```
def generate_launch_description():
    node = Node(
        package='pkg_helloworld_py',
        executable='helloworld',
    )
    return LaunchDescription([node])
```

Define a variable node as the return value for node startup. Call the Node function with two important parameters, package and executable.

- package: Represents the package, the name of the package.
- executable: Represents the program to execute, the name of the executable program.

Finally, call LaunchDescription function with node parameter to execute and return.

```
return LaunchDescription([node])
```

3. Writing Launch for Multiple Nodes

3.1. Create Launch File

Create a new [multi_node_launch.py] file and add the following content:

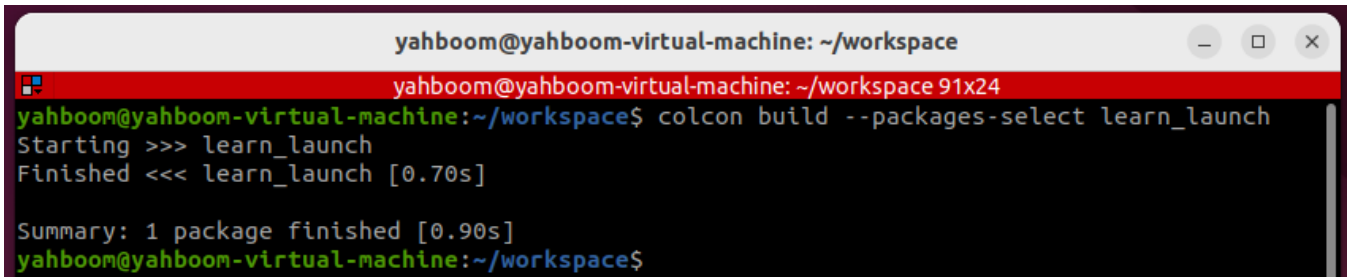
```
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    publisher_node = Node(
        package='pkg_topic',
        executable='publisher_demo',
        output='screen'
    )
    subscriber_node = Node(
        package='pkg_topic',
        executable='subscriber_demo',
        output='screen'
    )
    return LaunchDescription([
        publisher_node,
        subscriber_node
    ])
```

```
})
```

3.2. Build Package

```
colcon build --packages-select learn_launch
```



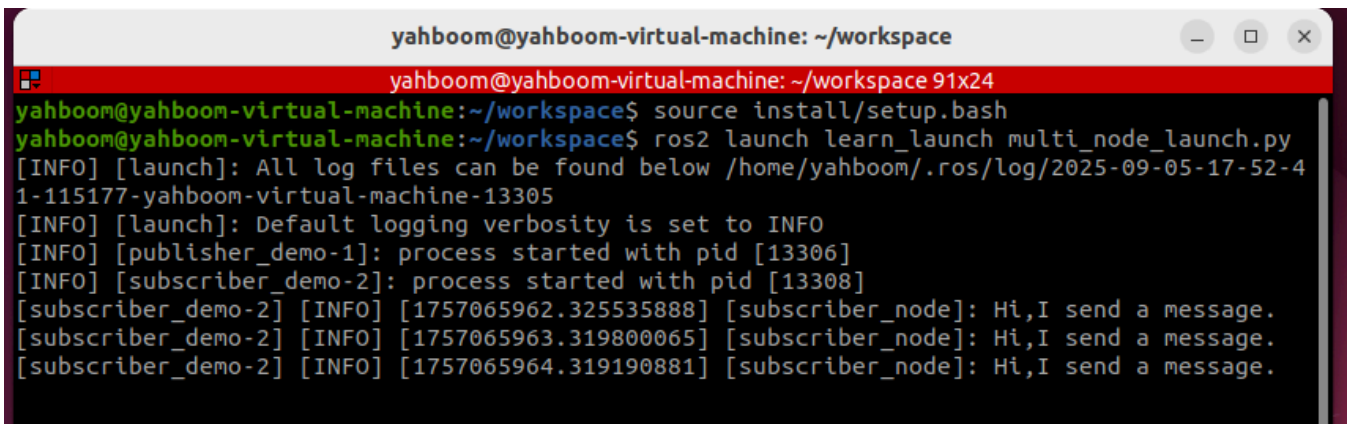
```
yahboom@yahboom-virtual-machine: ~/workspace
yahboom@yahboom-virtual-machine: ~/workspace 91x24
yahboom@yahboom-virtual-machine:~/workspace$ colcon build --packages-select learn_launch
Starting >>> learn_launch
Finished <<< learn_launch [0.70s]

Summary: 1 package finished [0.90s]
yahboom@yahboom-virtual-machine:~/workspace$
```

3.3. Run Program

- Refresh environment variables, then run the launch file:

```
ros2 launch learn_launch multi_node_launch.py
```



```
yahboom@yahboom-virtual-machine: ~/workspace
yahboom@yahboom-virtual-machine: ~/workspace 91x24
yahboom@yahboom-virtual-machine:~/workspace$ source install/setup.bash
yahboom@yahboom-virtual-machine:~/workspace$ ros2 launch learn_launch multi_node_launch.py
[INFO] [launch]: All log files can be found below /home/yahboom/.ros/log/2025-09-05-17-52-41-115177-yahboom-virtual-machine-13305
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [publisher_demo-1]: process started with pid [13306]
[INFO] [subscriber_demo-2]: process started with pid [13308]
[subscriber_demo-2] [INFO] [1757065962.325535888] [subscriber_node]: Hi,I send a message.
[subscriber_demo-2] [INFO] [1757065963.319800065] [subscriber_node]: Hi,I send a message.
[subscriber_demo-2] [INFO] [1757065964.319190881] [subscriber_node]: Hi,I send a message.
```

If the terminal doesn't print content, we can check which nodes are started to verify if startup was successful. Enter in the terminal:

```
ros2 node list
```

```
python3 x
○ root@unbutu:~/yahboomcar_ros2_ws/yahboomcar_ws# ros2 launch pkg_topic multi_node_launch.py
[INFO] [launch]: All log files can be found below /root/.ros/log/2023-10-31-04-00-24-504489-unbutu-107535
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [publisher_demo-1]: process started with pid [107555]
[INFO] [subscriber_demo-2]: process started with pid [107557]
█

问题 输出 调试控制台 终端 端口 3
● root@unbutu:~# ros2 node list
/publisher_node
/subscriber_node
○ root@unbutu:~# █
```

3.4. Source Code Analysis

It's roughly the same as `simple_node_launch.py`, just with one more node.

4. Topic Remapping Example

4.1. Create Launch File

Create a `[remap_name_launch.py]` file in the same directory as `multi_node_launch.py` and add the following content:

```
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    publisher_node = Node(
        package='pkg_topic',
        executable='publisher_demo',
        output='screen',
        remappings=[("/topic_demo", "/topic_update")]
    )
    return LaunchDescription([
        publisher_node
    ])
```

4.2. Build Package

```
colcon build --packages-select learn_launch
```

```
yahboom@yahboom-virtual-machine: ~/workspace
yahboom@yahboom-virtual-machine: ~/workspace 91x24
yahboom@yahboom-virtual-machine:~/workspace$ colcon build --packages-select learn_launch
Starting >>> learn_launch
Finished <<< learn_launch [0.70s]

Summary: 1 package finished [0.90s]
yahboom@yahboom-virtual-machine:~/workspace$
```

4.3. Run Program

First, let's see what topic the publisher_demo node publishes before remapping:

```
ros2 launch learn_launch multi_node_launch.py
ros2 topic list
```

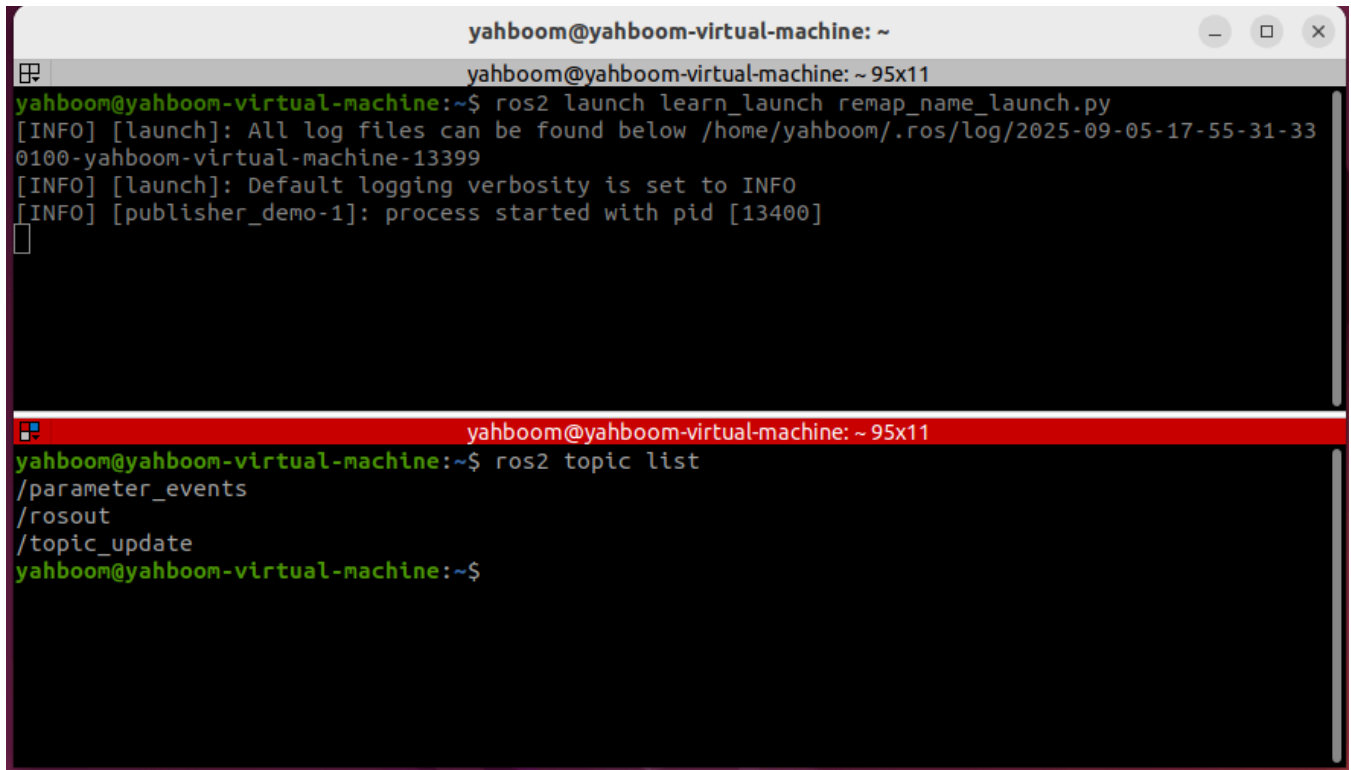
```
python3 x
root@ubuntu:~/yahboomcar_ros2_ws/yahboomcar_ws# ros2 launch pkg_topic multi_node_launch.py
[INFO] [launch]: All log files can be found below /root/.ros/log/2023-10-31-04-11-51-488377-ubuntu-115135
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [publisher_demo-1]: process started with pid [115137]
[INFO] [subscriber_demo-2]: process started with pid [115139]
█

问题 输出 调试控制台 终端 端口 3
● root@ubuntu:~/yahboomcar_ros2_ws/yahboomcar_ws# ros2 topic list
/parameter_events
/rosout
/topic_demo
root@ubuntu:~/yahboomcar_ros2_ws/yahboomcar_ws#
```

Here the topic is [/topic_demo]

- Refresh environment variables again and run the program after topic remapping to see the changes:

```
ros2 launch learn_launch remap_name_launch.py
ros2 topic list
```



The image shows two terminal windows from a virtual machine named 'yahboom@yahboom-virtual-machine'. The top window shows the execution of `ros2 launch learn_launch remap_name_launch.py`, which outputs log messages indicating that log files are located at `/home/yahboom/.ros/log/2025-09-05-17-55-31-330100-yahboom-virtual-machine-13399`, the logging verbosity is set to INFO, and a process named `publisher_demo-1` has started with PID 13400. The bottom window shows the execution of `ros2 topic list`, which outputs the following topics: `/parameter_events`, `/rosout`, and `/topic_update`.

```
yahboom@yahboom-virtual-machine: ~  
yahboom@yahboom-virtual-machine: ~ 95x11  
yahboom@yahboom-virtual-machine:~$ ros2 launch learn_launch remap_name_launch.py  
[INFO] [launch]: All log files can be found below /home/yahboom/.ros/log/2025-09-05-17-55-31-330100-yahboom-virtual-machine-13399  
[INFO] [launch]: Default logging verbosity is set to INFO  
[INFO] [publisher_demo-1]: process started with pid [13400]  
  
yahboom@yahboom-virtual-machine:~$ ros2 topic list  
/parameter_events  
/rosout  
/topic_update  
yahboom@yahboom-virtual-machine:~$
```

As shown above, the topic has been remapped to `/topic_update`.

4.4. Source Code Analysis

The main addition is:

```
remappings=[("/topic_demo", "/topic_update")]
```

This remaps the original `/topic_demo` topic to `/topic_update`.

5. Launch File Nesting - Include Another Launch File Example

5.1. Create Launch File

Create an `[include_launch.py]` file in the same directory as `multi_node_launch.py` and add the following content:

```
from launch import LaunchDescription  
from launch_ros.actions import Node  
import os  
from launch.actions import IncludeLaunchDescription  
from launch.launch_description_sources import PythonLaunchDescriptionSource  
from ament_index_python.packages import get_package_share_directory  
  
def generate_launch_description():  
    hello_launch = IncludeLaunchDescription(PythonLaunchDescriptionSource(  

```

```

    [os.path.join(get_package_share_directory('learn_launch'), 'launch'),
      '/multi_node_launch.py']],
  )
  return LaunchDescription([
    hello_launch
  ])

```

5.2. Build Package

```
colcon build --packages-select learn_launch
```

```

yahboom@yahboom-virtual-machine: ~/workspace
yahboom@yahboom-virtual-machine: ~/workspace 91x24
yahboom@yahboom-virtual-machine:~/workspace$ colcon build --packages-select learn_launch
Starting >>> learn_launch
Finished <<< learn_launch [0.70s]

Summary: 1 package finished [0.90s]
yahboom@yahboom-virtual-machine:~/workspace$

```

5.3. Run Program

- Refresh environment variables and run the launch file:

```
ros2 launch learn_launch include_launch.py
```

```

yahboom@yahboom-virtual-machine: ~/workspace
yahboom@yahboom-virtual-machine: ~/workspace 104x25
yahboom@yahboom-virtual-machine:~/workspace$ source install/setup.bash
yahboom@yahboom-virtual-machine:~/workspace$ ros2 launch learn_launch include_launch.py
[INFO] [launch]: All log files can be found below /home/yahboom/.ros/log/2025-09-05-18-03-499053-yahb
oom-virtual-machine-13863
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [publisher_demo-1]: process started with pid [13864]
[INFO] [subscriber_demo-2]: process started with pid [13866]
[subscriber_demo-2] [INFO] [1757066584.707313583] [subscriber_node]: Hi,I send a message.
[subscriber_demo-2] [INFO] [1757066585.698788303] [subscriber_node]: Hi,I send a message.
[subscriber_demo-2] [INFO] [1757066586.698394059] [subscriber_node]: Hi,I send a message.
[subscriber_demo-2] [INFO] [1757066587.698883703] [subscriber_node]: Hi,I send a message.
[subscriber_demo-2] [INFO] [1757066588.698925104] [subscriber_node]: Hi,I send a message.
[subscriber_demo-2] [INFO] [1757066589.697920534] [subscriber_node]: Hi,I send a message.

```

5.4. Source Code Analysis

- Nesting launch files requires using two classes from the launch system: IncludeLaunchDescription and PythonLaunchDescriptionSource
- `os.path.join(get_package_share_directory('learn_launch'))`: Get the location of the package, where 'learn_launch' is the package name;

- 'launch'): Represents the folder where launch files are stored under the package;
- /multi_node_launch.py': Represents the /multi_node_launch.py file under the package's launch folder.

6. Comprehensive Launch File Example

This example mainly demonstrates how to write complex launch files. The program functionality can be ignored.

6.1. Create Launch File

Create a [complex_launch.py] file in the same directory as multi_node_launch.py and add the following content:

```
import os
from ament_index_python import get_package_share_directory
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument
from launch.actions import IncludeLaunchDescription
from launch.actions import GroupAction
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch.substitutions import LaunchConfiguration
from launch.substitutions import TextSubstitution
from launch_ros.actions import Node
from launch_ros.actions import PushRosNamespace

def generate_launch_description():

    # args that can be set from the command line or a default will be used
    background_r_launch_arg = DeclareLaunchArgument(
        "background_r", default_value=TextSubstitution(text="0")
    )
    background_g_launch_arg = DeclareLaunchArgument(
        "background_g", default_value=TextSubstitution(text="255")
    )
    background_b_launch_arg = DeclareLaunchArgument(
        "background_b", default_value=TextSubstitution(text="0")
    )
    chatter_ns_launch_arg = DeclareLaunchArgument(
        "chatter_ns", default_value=TextSubstitution(text="my/chatter/ns")
    )

    # include another launch file
    launch_include = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(
            os.path.join(
                get_package_share_directory('demo_nodes_cpp'),
                'launch/topics/talker_listener_launch.py')
        )
    )

    # include another launch file in the chatter_ns namespace
```

```

launch_include_with_namespace = GroupAction(
    actions=[
        # push-ros-namespace to set namespace of included nodes
        PushRosNamespace(LaunchConfiguration('chatter_ns')),
        IncludeLaunchDescription(
            PythonLaunchDescriptionSource(
                os.path.join(
                    get_package_share_directory('demo_nodes_cpp'),
                    'launch/topics/talker_listener_launch.py'))
            ),
    ]
)

# start a turtlesim_node in the turtlesim1 namespace
turtlesim_node = Node(
    package='turtlesim',
    namespace='turtlesim1',
    executable='turtlesim_node',
    name='sim'
)

# start another turtlesim_node in the turtlesim2 namespace
# and use args to set parameters
turtlesim_node_with_parameters = Node(
    package='turtlesim',
    namespace='turtlesim2',
    executable='turtlesim_node',
    name='sim',
    parameters=[{
        "background_r": LaunchConfiguration('background_r'),
        "background_g": LaunchConfiguration('background_g'),
        "background_b": LaunchConfiguration('background_b'),
    }]
)

# perform remap so both turtles listen to the same command topic
forward_turtlesim_commands_to_second_turtlesim_node = Node(
    package='turtlesim',
    executable='mimic',
    name='mimic',
    remappings=[
        ('/input/pose', '/turtlesim1/turtle1/pose'),
        ('/output/cmd_vel', '/turtlesim2/turtle1/cmd_vel'),
    ]
)

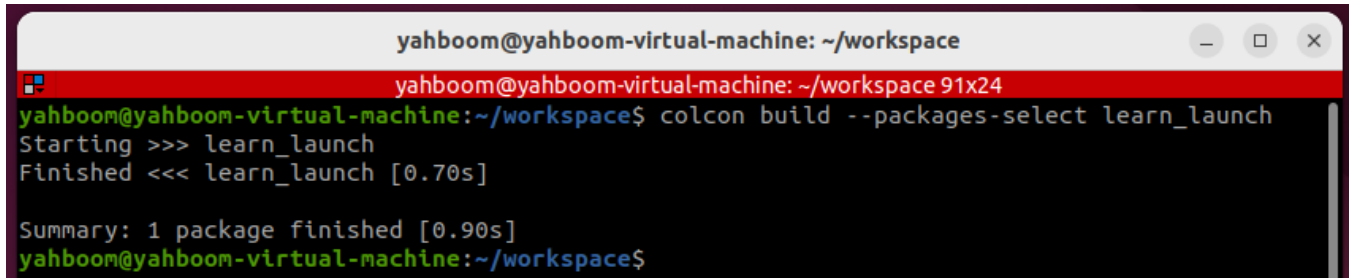
return LaunchDescription([
    background_r_launch_arg,
    background_g_launch_arg,
    background_b_launch_arg,
    chatter_ns_launch_arg,
    launch_include,
    launch_include_with_namespace,

```

```
turtlesim_node,  
turtlesim_node_with_parameters,  
forward_turtlesim_commands_to_second_turtlesim_node,  
])
```

6.2. Build Workspace

```
colcon build --packages-select learn_launch
```

A terminal window titled 'yahboom@yahboom-virtual-machine: ~/workspace' with standard window controls. The prompt is 'yahboom@yahboom-virtual-machine: ~/workspace 91x24'. The user enters 'colcon build --packages-select learn_launch'. The output shows 'Starting >>> learn_launch', 'Finished <<< learn_launch [0.70s]', and a 'Summary: 1 package finished [0.90s]'. The prompt returns to 'yahboom@yahboom-virtual-machine: ~/workspace\$'.

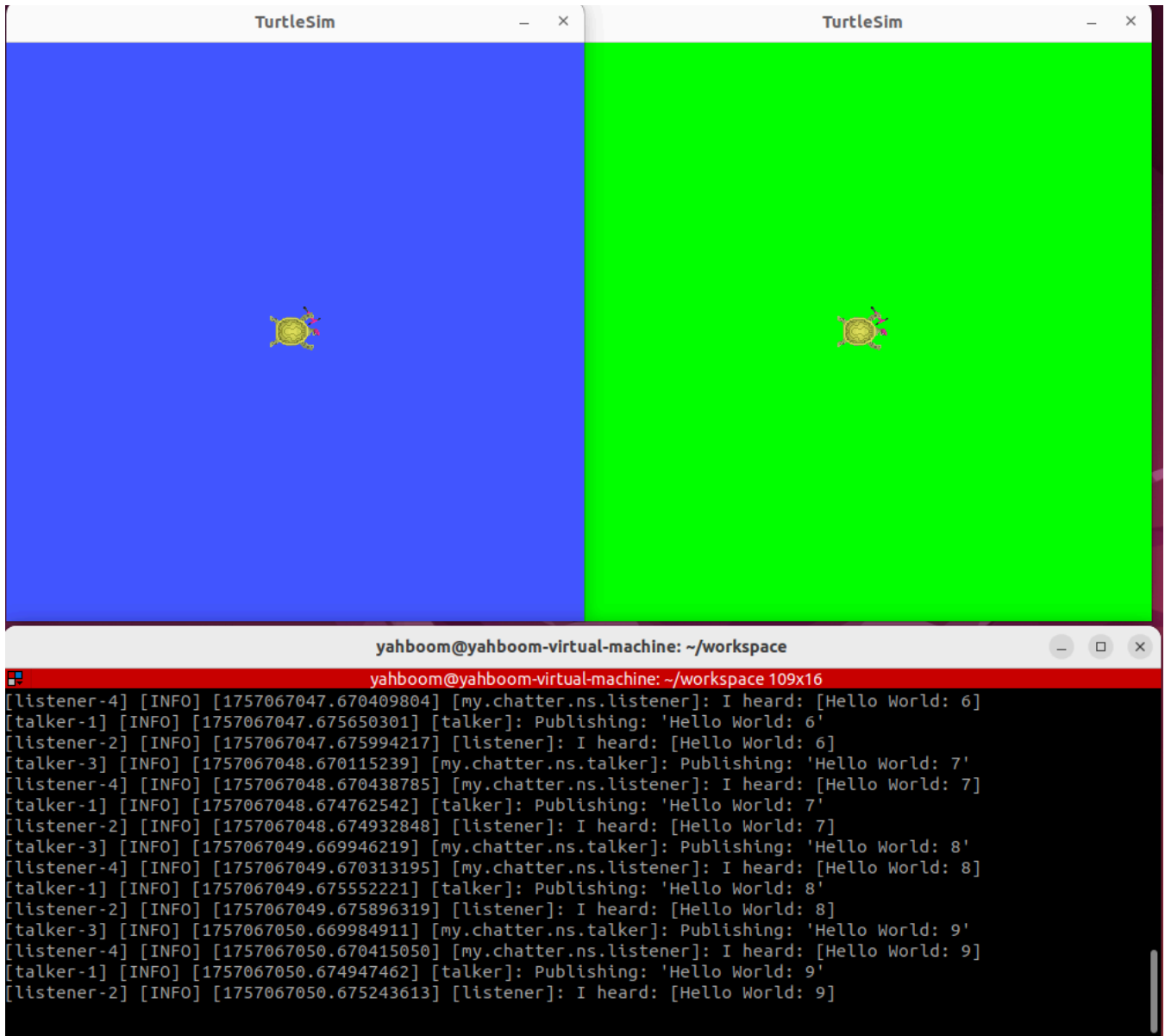
```
yahboom@yahboom-virtual-machine: ~/workspace  
yahboom@yahboom-virtual-machine: ~/workspace 91x24  
yahboom@yahboom-virtual-machine:~/workspace$ colcon build --packages-select learn_launch  
Starting >>> learn_launch  
Finished <<< learn_launch [0.70s]  
  
Summary: 1 package finished [0.90s]  
yahboom@yahboom-virtual-machine:~/workspace$
```

6.3. Run Program

- Refresh environment variables in the terminal and run the launch file:

```
ros2 launch learn_launch complex_launch.py
```

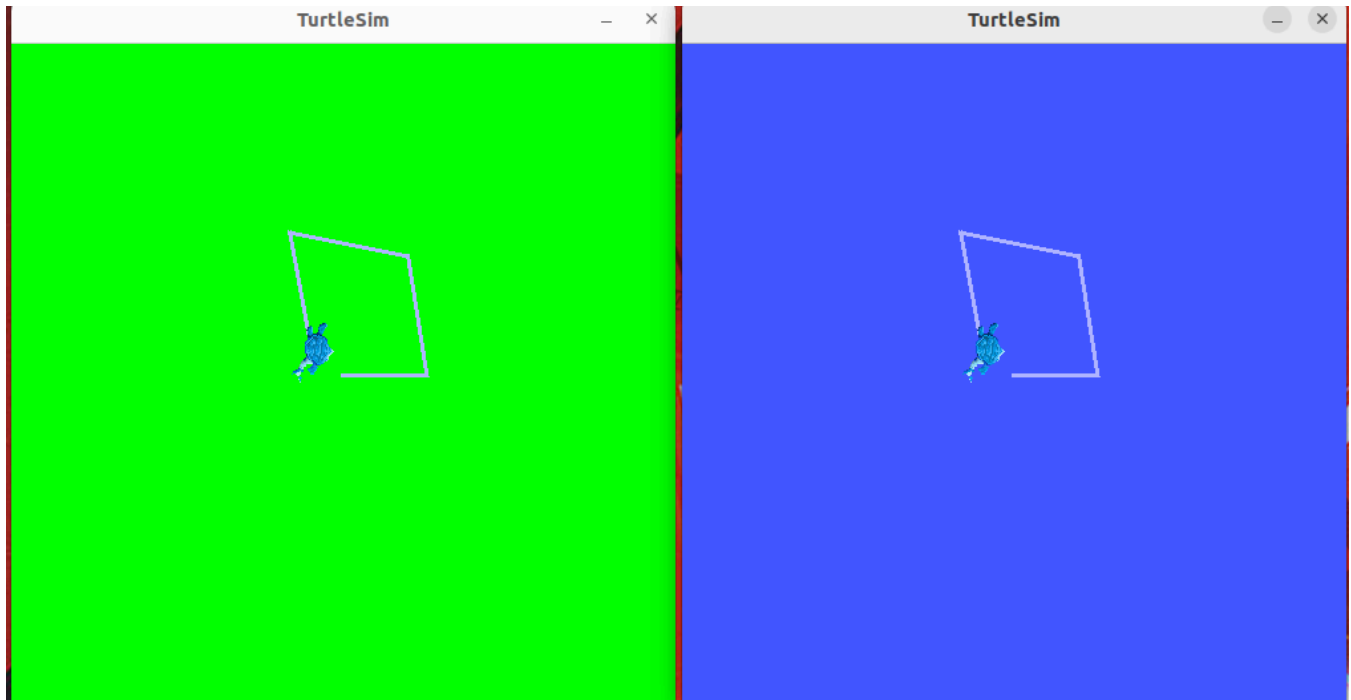
Two turtles will appear on the host machine's VNC.



- Start the keyboard control node and add namespace (because we added namespace when starting nodes in the launch file):

```
ros2 run turtlesim turtle_teleop_key --ros-args -r __ns:=/turtlesim1
```

- Use arrow keys to control turtle 1's movement. Turtle 2 will completely mimic turtle 1's behavior.



6.4. Program Description

The program mainly starts:

1. demo_nodes_cpp's talker_listener node,
2. talker_listener node with namespace,
3. Turtle 1 with turtlesim1 as namespace,
4. Turtle 2 with turtlesim2 as namespace,
5. Perform remapping so both turtles listen to the same command topic.

7. XML Implementation

7.1. Create Launch File

Create a [complex_launch.xml] file in the same directory as complex_launch.py and add the following content:

```
<launch>

  <!-- args that can be set from the command line or a default will be used -->
  <arg name="background_r" default="0"/>
  <arg name="background_g" default="255"/>
  <arg name="background_b" default="0"/>
  <arg name="chatter_ns" default="my/chatter/ns"/>

  <!-- include another launch file -->
  <include file="$(find-pkg-share
demo_nodes_cpp)/launch/topics/talker_listener_launch.py"/>
  <!-- include another launch file in the chatter_ns namespace-->
  <group>
    <!-- push-ros-namespace to set namespace of included nodes -->
```

```

    <push-ros-namespace namespace="$(var chatter_ns)"/>
    <include file="$(find-pkg-share
demo_nodes_cpp)/launch/topics/talker_listener_launch.py"/>
  </group>

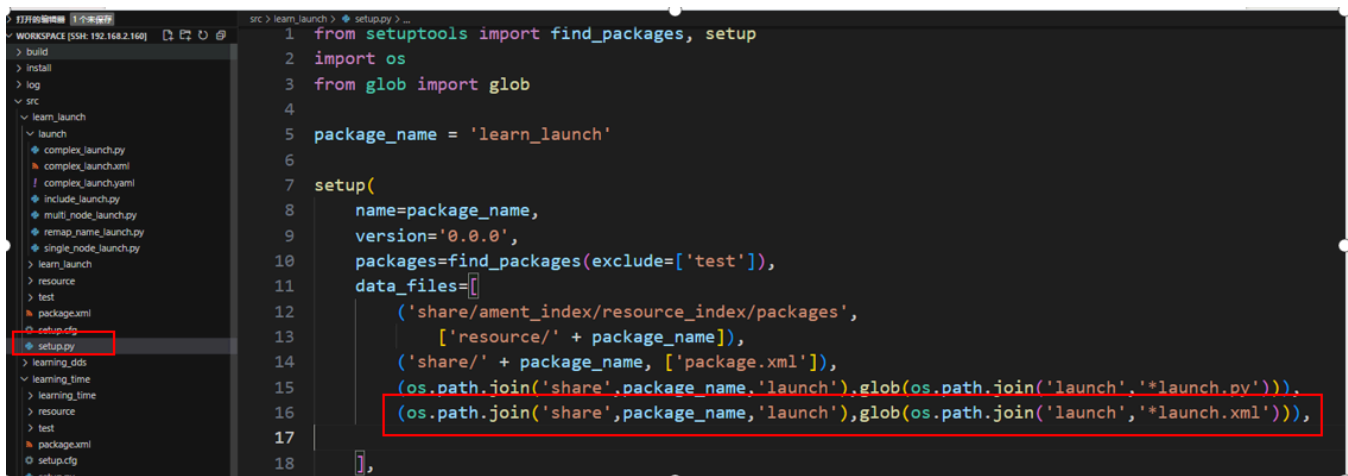
  <!-- start a turtlesim_node in the turtlesim1 namespace -->
  <node pkg="turtlesim" exec="turtlesim_node" name="sim" namespace="turtlesim1"/>
  <!-- start another turtlesim_node in the turtlesim2 namespace
    and use args to set parameters -->
  <node pkg="turtlesim" exec="turtlesim_node" name="sim" namespace="turtlesim2">
    <param name="background_r" value="$(var background_r)"/>
    <param name="background_g" value="$(var background_g)"/>
    <param name="background_b" value="$(var background_b)"/>
  </node>

  <!-- perform remap so both turtles listen to the same command topic -->
  <node pkg="turtlesim" exec="mimic" name="mimic">
    <remap from="/input/pose" to="/turtlesim1/turtle1/pose"/>
    <remap from="/output/cmd_vel" to="/turtlesim2/turtle1/cmd_vel"/>
  </node>
</launch>

```

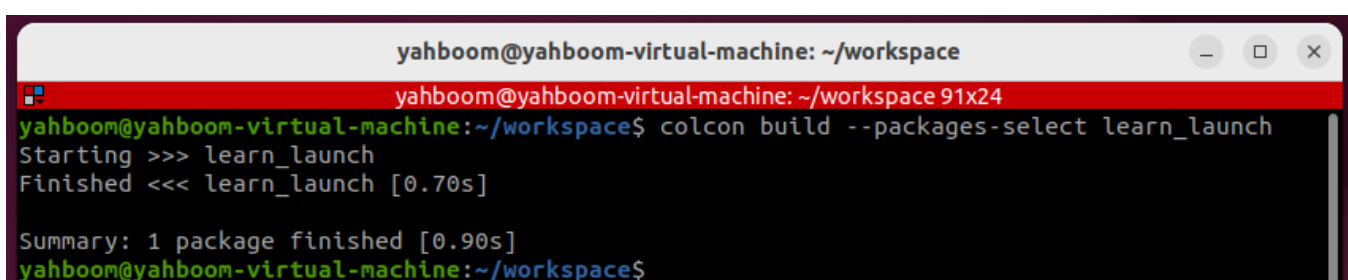
7.2. setup.py File Configuration

- You need to configure the build file so that during build, our .xml format launch file is copied to the install directory, so the ROS system can find our file.



7.3. Build Package

```
colcon build --packages-select learn_launch
```

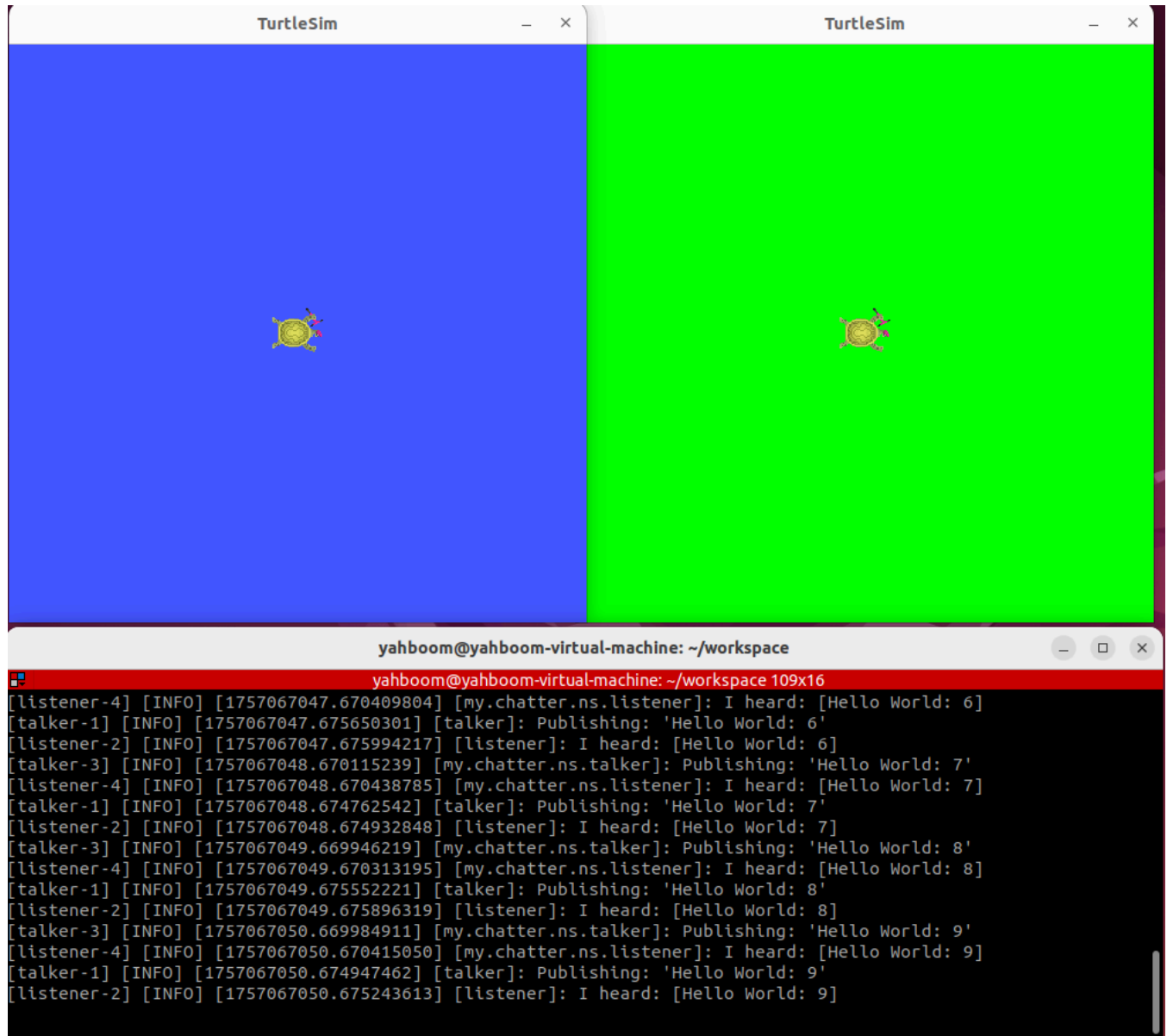


7.4. Run Program

Enter in terminal:

```
ros2 launch learn_launch complex_launch.xml
```

- As expected, two turtles will appear and the terminal will print log information.



- Start the keyboard control node and add namespace:

```
ros2 run turtlesim turtle_teleop_key --ros-args -r __ns:=/turtlesim1
```

Use keyboard to control and start turtle 1. Turtle 2 will completely mimic turtle 1's behavior.

8. YAML Implementation

8.1. Create Launch File

Create a [complex_launch.yaml] file in the same directory as complex_launch.py and add the following content:

```
launch:

# args that can be set from the command line or a default will be used
- arg:
  name: "background_r"
  default: "0"
- arg:
  name: "background_g"
  default: "255"
- arg:
  name: "background_b"
  default: "0"
- arg:
  name: "chatter_ns"
  default: "my/chatter/ns"

# include another launch file
- include:
  file: "$(find-pkg-share demo_nodes_cpp)/launch/topics/talker_listener_launch.py"

# include another launch file in the chatter_ns namespace
- group:
  - push-ros-namespace:
    namespace: "$(var chatter_ns)"
  - include:
    file: "$(find-pkg-share demo_nodes_cpp)/launch/topics/talker_listener_launch.py"

# start a turtlesim_node in the turtlesim1 namespace
- node:
  pkg: "turtlesim"
  exec: "turtlesim_node"
  name: "sim"
  namespace: "turtlesim1"

# start another turtlesim_node in the turtlesim2 namespace and use args to set parameters
- node:
  pkg: "turtlesim"
  exec: "turtlesim_node"
  name: "sim"
  namespace: "turtlesim2"
  param:
  -
    name: "background_r"
    value: "$(var background_r)"
  -
```

```

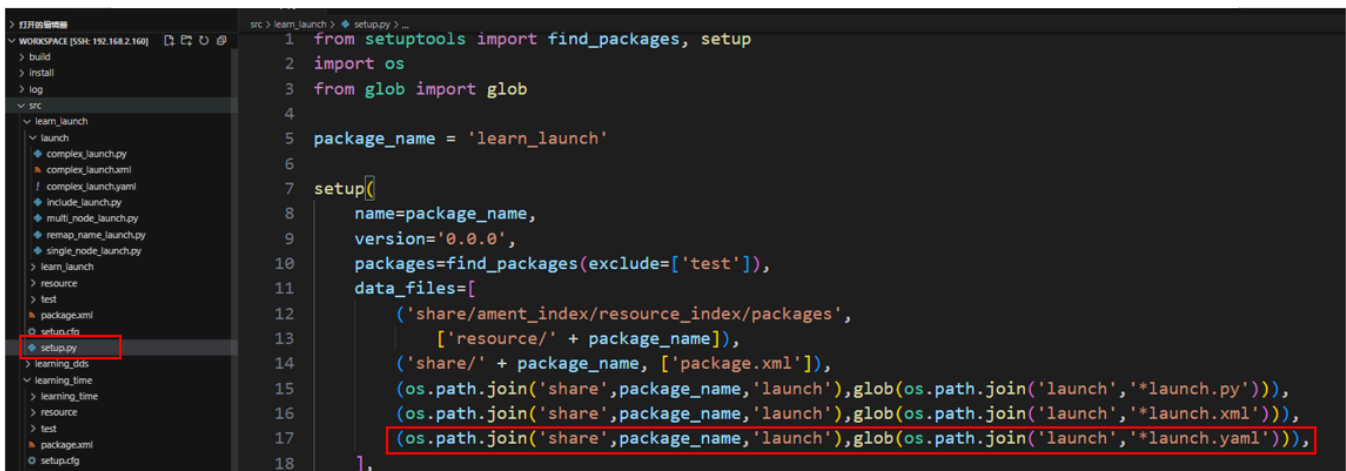
    name: "background_g"
    value: "${var background_g}"
-
    name: "background_b"
    value: "${var background_b}"

# perform remap so both turtles listen to the same command topic
- node:
    pkg: "turtlesim"
    exec: "mimic"
    name: "mimic"
    remap:
    -
        from: "/input/pose"
        to: "/turtlesim1/turtle1/pose"
    -
        from: "/output/cmd_vel"
        to: "/turtlesim2/turtle1/cmd_vel"

```

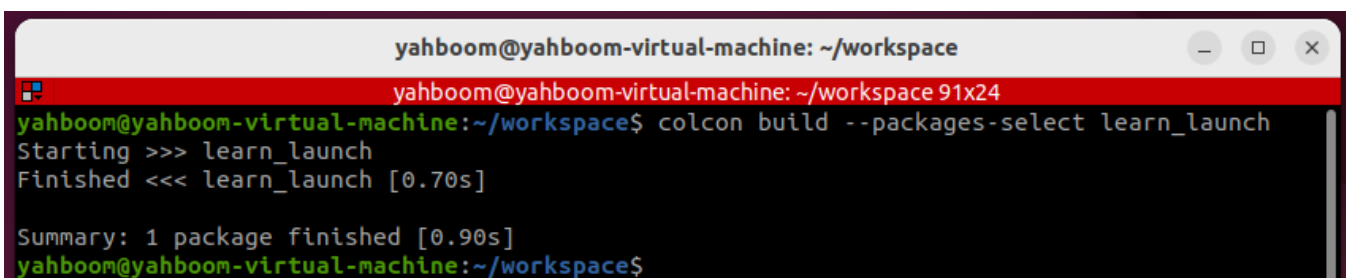
8.2. Configuration

- You need to configure the build file so that during build, our .yaml format launch file is copied to the install directory, so the ROS system can find our file.



8.3. Build Package

```
colcon build --packages-select learn_launch
```

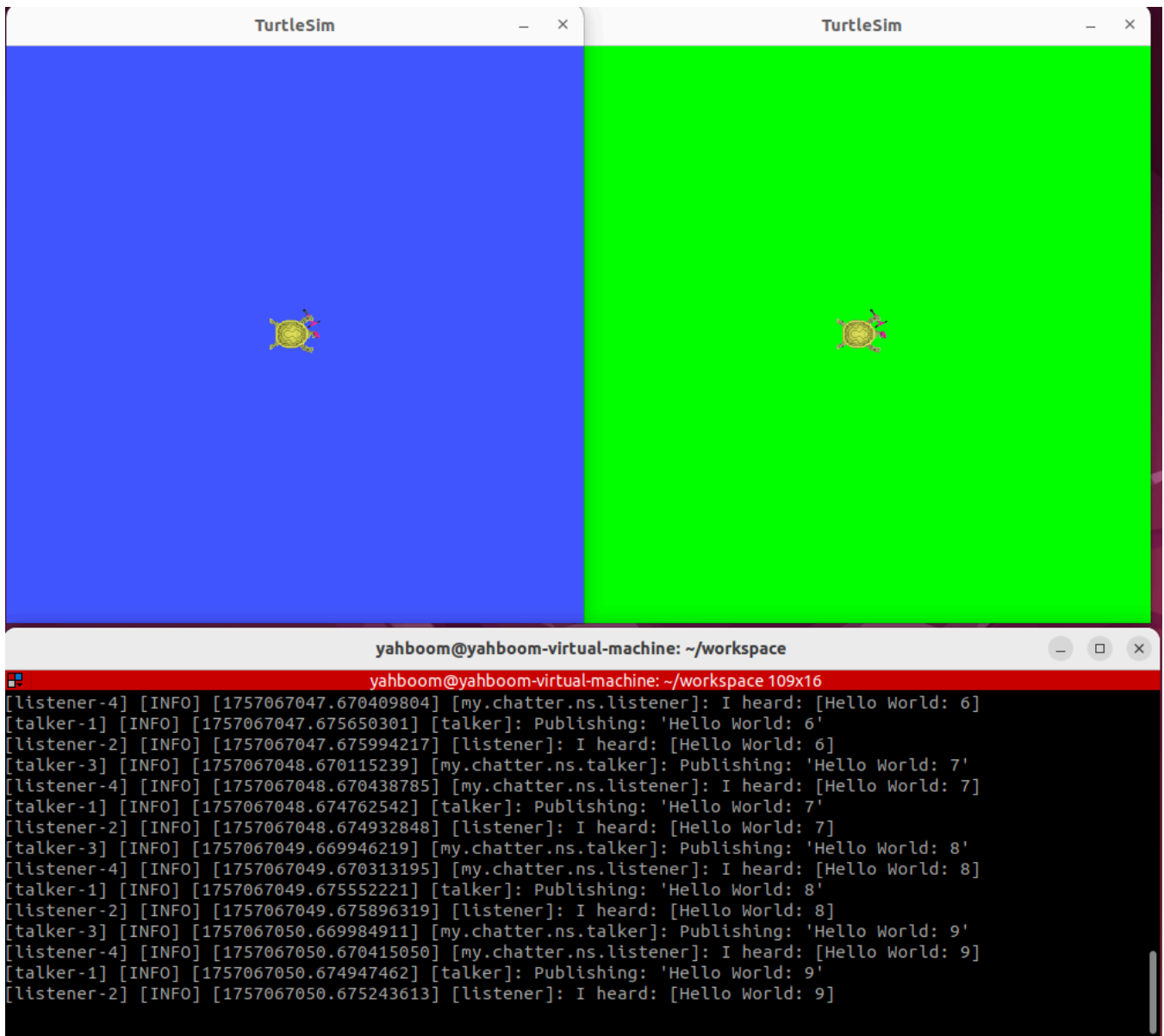


8.4. Run Program

- Refresh environment variables then run:

```
ros2 launch learn_launch complex_launch.yaml
```

- As expected, two turtles will appear and the terminal will print log information.



- Start the keyboard control node and add namespace:

```
ros2 run turtlesim turtle_teleop_key --ros-args -r __ns:=/turtlesim1
```

Use keyboard to control and start turtle 1. Turtle 2 will completely mimic turtle 1's behavior.